

Tool Command Language

Table of Content

- Basics (Arrays, List, Strings, Hashes, Loops).
- Functions (Procs).
- File Handling (File reading/writing).
- Pattern Matching.
- Real World related tasks in tcl.

TCL Basics

The first program

```
puts "hello world"; #puts is for printing  
#in command line
```

Output:

```
hello world
```

Without semicolon, we cannot add comment in same line. It will be syntax error

Semicolon (;) is not required in one single line of statement.

```
puts "hello world";  
puts "hello world without semicolon"
```

Output:

```
hello world
```

```
hello world without semicolon
```

Semicolon (;) is significant while writing two commands in single line.

```
puts "hello world"; puts "hello world without semicolon"  
#above statement is valid
```

```
puts "hello world" puts "hello world with semicolon";  
#above statement is not valid and will throw syntax error
```

Semicolon marks the end of statement.

Semicolon (;) is also significant if you want to put comment after the end of statement.

```
puts "hello world"    #comment
```

Above statement will throw syntax error

```
puts "hello world";    #comment after semicolon
```

Above statement is correct

Variable declaration:

Input:

```
set str "this is str";  
set value 1.5;  
puts "$str";  
puts "the value is $value";
```

Output:

```
this is str  
this is value: 1.5
```

Dollar '\$' is used as prefix with variable to print variable value in command line.

Example

Input

```
set var1 "Hello World"  
puts "The string is $var1"
```

Output

The string is Hello World

Square Braces

Square braces "[]" are used for assigning value to variables from other expressions.

Following is one example

```
set a "5";  
set b $a;           #this statement will assign b to a
```

- Below statement will also assign b to a but in single statement
set b [set a "5"];
- It will take statement inside square braces, evaluate it, and, assign output of it to the variable b.

Curly Braces

Curly Braces

Why the curly brace '{' is required when double quote is already there for printing ?

Curly braces doesn't recognize variables. It prints all the \$variables as it is.



<u>Input</u>	<u>Output</u>
<pre>set value 1.5; puts {the value is \$value}; puts "the value is \$value";</pre>	<pre>the value is \$value the value is 1.5</pre>

Example:

Input:

```
set a 5;  
set b 6;  
set c [expr $a+$b]  
puts $c  
puts "The answer is $c"  
puts {The answer is $c}
```

Output:

```
11  
The answer is 11  
The answer is $c
```

Arithmetic, Bitwise and Logical Operations

Arithmetic Operation

Expr Statement:

Expr is used to evaluate the expression.

Syntax:

Expr variable1 operator variable2

```
set a "5";
set b "3";
set c [expr "$a + $b"];
set d [expr "$a - $b"];
set e [expr "$a * $b"];
puts "addition of a and b is $c";      addition of a and b is 8
puts "subtraction of a and b is $d";    subtraction of a and b is 2
puts "multiply of a and b is $e";       multiply of a and b is 15
```

Expr Statement:

```
expr 10+5      ->      15
expr 10.0+5    ->      15.0
expr 12/7      ->      1
expr 12/7+3.0  ->      4.0
expr 12.0/7+3.0 ->      4.71428
```

Integers are used until a floating point number is introduced, after which floating-point number is used.

Logical Operation

Logical Operations

variable A holds 1

variable B holds 0

Operator	Description	Example
&&	Logical AND operator	(A && B) is false.
	Logical OR operator	(A B) is true.
!	Logical NOT operator	!(A && B) is true.

Bitwise Operations

Bitwise operator works on bits and perform bit-by-bit operation.

& = Bitwise AND

| = Bitwise OR

^ = Bitwise XOR

Assume if A = 60; and B = 13. Now, in binary format they will be as follows –

A = 0011 1100

B = 0000 1101

A&B = 0000 1100

A|B = 0011 1101

A^B = 0011 0001

Bitwise Operations

Bitwise operator works on bits and perform bit-by-bit operation.

& = Bitwise AND

| = Bitwise OR

^ = Bitwise XOR

A = 0011 1100

B = 0000 1101

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) will give 12, which is 0000 1100
	Binary OR Operator copies a bit if it exists in either operand.	(A B) will give 61, which is 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) will give 49, which is 0011 0001
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 will give 240, which is 1111 0000
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 will give 15, which is 0000 1111

Example

Input:

```
set a [expr 0b11001]
```

```
set b [expr 0b10101]
```

```
puts "The result is [expr $a & $b]"
```

Output:

The result is 17

Example

Input:

```
set a [expr 0b11001]
```

```
set b [expr 0b10101]
```

```
puts "The result is [expr $a && $b]"
```

Output

The result is 1

Note : Basically non zero value is considered as 1 in binary.

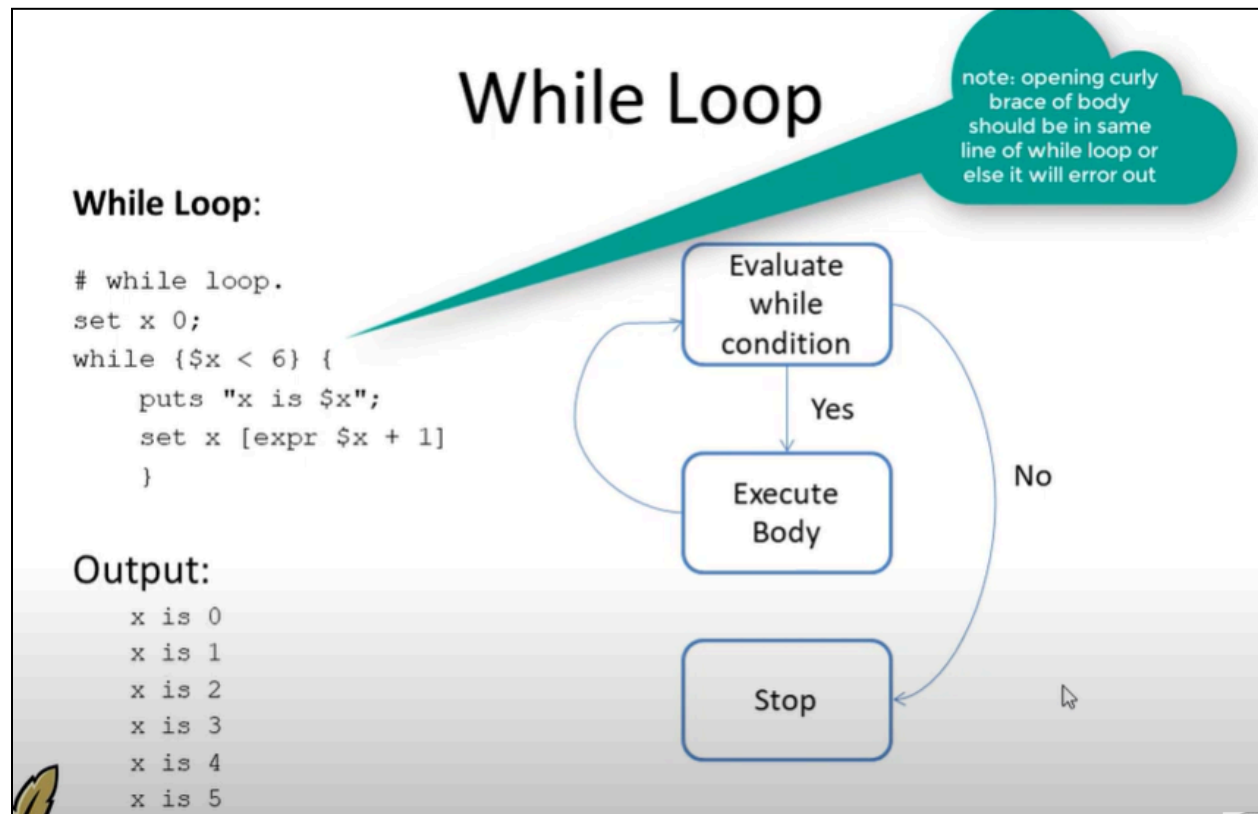
Conditions in TCL

If condition

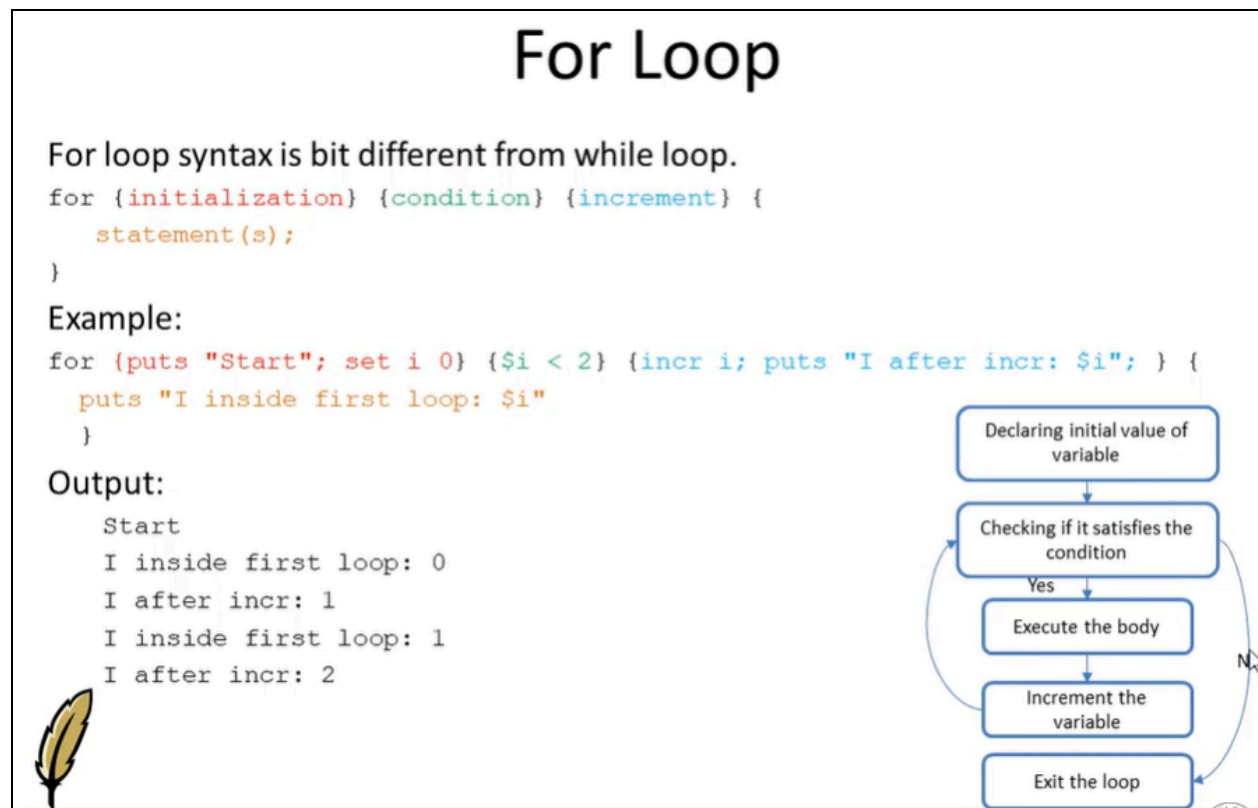
Example

Input:

While Loop



for Loop



Array and Loops

Arrays

Arrays

Array Declaration: Arrays are ordered set of values

The syntax is:

```
set ArrayName(Index) value
```

```
set institute(0) VLSI
set institute(1) Academy
set institute(2) India
```

Index	Value
0	VLSI
1	Academy
2	India

```
puts $institute(0)
puts $institute(1)
puts $institute(2)
```

Output:

```
VLSI
Academy
India
```

Arrays

Iterating over arrays: one can iterate/loop over arrays in tcl

```
set institute(0) VLSI
set institute(1) Academy
set institute(2) India
```

```
for {set index 0} {$index < [array size institute]} {incr index} {
    puts "institute($index) : $institute($index) "
}
```

Output:

```
institute(0) : VLSI
institute(1) : Academy
institute(2) : India
```

Associative Array

Associative Arrays

Associative Arrays: Also called as dictionaries.

- Associative arrays are unordered set of values which can be accessed with corresponding keys.
- Associative arrays have an index that is not necessarily a number, and can be sparsely populated.

- Example:

```
set Employee1(Name) "Jacob"  
set Employee1(Age) 24  
puts $Employee1(Name)  
puts $Employee1(Age)
```

Associative Arrays:

- Syntax of retrieving indices, is as follows:

```
[array names arrayName]
```

- Example:

```
set Employee1(Name) "Latha"  
set Employee1(Age) 24  
puts [array names Employee1]
```

- Output:

```
Age Name
```

Foreach Loop

Foreach Loop: foreach is more useful in the event of *associative arrays*.

```
set institute(0) VLSI
set institute(1) Academy
set institute(2) India
```

```
foreach index [array names institute] {
puts "institute($index):$institute($index)"
}
```

This will be expanded to all keys present in "institute" array

Output:

```
institute(0):VLSI
institute(1):Academy
institute(2):India
```

Looping through array

Executing shell commands and escape sequences

exec command

- It can be used to execute shell commands
- Example:

```
exec ls                #lists all files in directory
exec ls *.tcl          #lists all .tcl files in directory
```

Data Structure in TCL
String Operation in TCL

String commands

- **string compare** *string1 string2*

Compares string1 and string2 lexicographically. Returns 0 if equal, -1 if string1 comes before string2, else 1.

```
puts [string compare "Golden" "Light"]
```

 Output: -1

- **string index** *string index*

Returns the character at index

```
puts [string index "Timing Path" 4]
```

 Output: n

- **string length** *string*

Returns length of string

```
puts [string length "Timing Path"]
```

 output: 11

String commands

- **string range** *string index1 index2*

Return the range of characters in string from index1 to index2

```
puts [string range "I am studying physical design" 2 12]
```

output: am studying

- **string tolower** *string*

Returns the lowercase string

```
puts [string tolower "VLSI DESIGN"]
```

Output: vlsi design

- **string toupper** *string*

Returns the uppercase string

```
puts [string toupper "vlsi design"]
```

Output: VLSI DESIGN

String commands

- **trimright string ?trimcharacters?**

Removes trimcharacters in left end of string. The default trimcharacters is whitespace.

```
set s1 "VLSI Academy India Pvt Ltd"
set s2 "Pvt Ltd"
puts "Trim right: \"$s2\" in \"$s1\""
puts "new string: \"[string trimright $s1 $s2]\""
```

Output:

```
Trim right: "Pvt Ltd" in "VLSI Academy India Pvt Ltd"
new string: "VLSI Academy India"
```

String commands

- **trimleft string ?trimcharacters?**

Removes trimcharacters in left beginning of string. The default trimcharacters is whitespace.

```
set s1 "Physical Design includes stages from floorplan to routing"
set s2 "Physical"
puts "Trim left: \"$s2\" in \"$s1\""
puts "new string: \"[string trimleft $s1 $s2]\""
```

Output:

```
Trim left: "Physical" in "Physical Design includes stages from
floorplan to routing"
new string: "Design includes stages from floorplan to routing"
```

String commands

- **trim string ?trimcharacters?**

Removes trimcharacters in both ends of string. The default trim characters is whitespace

```
set s1 ":::Physical Design:::"
set s2 ":"
puts "Trim on both sides- \"$s2\" in \"$s1\""
puts "new string: \"[string trim $s1 $s2]\""
```

Output:

```
Trim on both sides- ":" in ":::Physical Design:::"
new string: "Physical Design"
```

String commands

- **match pattern string**

Returns 1 if the string matches the pattern.

```
set s1 "test-id@goldenLightVLSI.com"
set s2 "*@*.com"
puts "Matching pattern $s2 in $s1"
if { [string match "@*.com" $s1] } {
  puts "match found"
} else { puts "match not found" }

puts "Matching pattern \"tcl\" in $s1"
if { [string match {tcl} $s1] } {
  puts "match found"
} else { puts "match not found" }
```

Output:

```
Matching pattern *@*.com in test-
id@goldenLightVLSI.com
```

```
match found
```

```
Matching pattern "tcl" in test-
id@goldenLightVLSI.com
```

```
match not found
```

String commands

- **Appending string:**

It appends string to original string itself. It doesn't create a new string.

Syntax:

```
append original_string appending_string
```

Example:

```
set s1 "We love to study tcl"  
append s1 "by VLSI Academy"  
puts $s1
```

Output:

```
We love to study tcl by VLSI Academy
```

List in TCL

List Operations

Concat

- Concatenates the args into a single list.
For example: set c [concat \$a \$b]

Linsert

- Inserts the items into list at specified index and returns the new list with inserted item
For e.g.: set c [linsert "a b c" 1 "z"]

Lreplace

- Replaces the list item at specified index with new specified item.
For e.g. set c [lreplace "a b c" 1 "z"]

List Operations

Lappend

- Appends new item at the last, to the existing list

Lsort

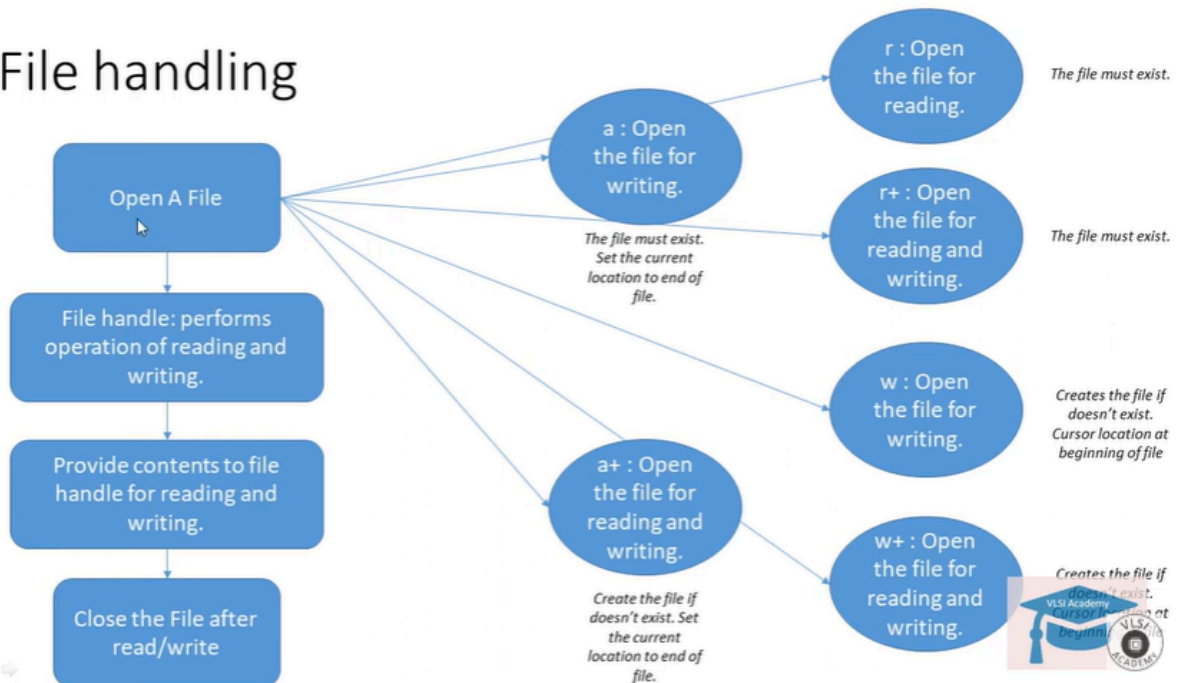
- Sorts list and returns a new list in the sorted order. By default, sorts the list in dictionary order

Lrange

- Returns a list composed of the first through last entries in the list

File Handling in TCL

File handling



File handling

Reading entire file at once

```
set fp [open "input.txt" r]
set file_data [read $fp]
puts $file_data
close $fp
```

Reading file line by line

```
set fp [open "input.txt" r]
while { [gets $fp data] >= 0 } {
    puts $data
}
close $fp
```

File handling

Writing file contents

```
set fp [open "input.txt" w+]
puts $fp "test"
close $fp
```

Procedures or Functions in TCL

Procs : Functions in tcl

```
proc welcomeProc {} {
    puts "Hi, This is the first basic of tcl!"
}
welcomeProc
```

Calling a proc in tcl.
Simply write down
the name of proc with
arguments, if any.

Example

Input:

```
proc text_print {} {
    puts "Hello World!"
}
```

```
}  
puts "[text_print]"
```

Output:

Hello World!

Example

Input:

```
proc comp {a b} {  
  if {$a > $b} {  
    return $a  
  } else {  
    return $b  
  }  
}  
puts "The result is : [comp 10 15]"
```

Output:

The result is : 15

Procs with multiple arguments

```
proc sum {a b} {  
  return [expr $a+$b]  
}  
puts [sum 10 30]
```

In this case number of arguments is known in advance and is fixed count.

Example

Input:

```
proc sum {a b} {  
  return [expr $a + $b]  
}  
puts "sum of two number : [sum 10 15]"
```

Output:

sum of two number : 25

Procs with multiple arguments

```
proc employee_modifier {emp_ID} {  
    return [join [list "1" $emp_ID] ""]  
}  
puts [employee_modifier 7892]
```

Output : 17892

Example

Input:

```
proc mul {a b} {  
    return [expr $a * $b]  
}
```

puts "Multiplication of two number : [mul 5 6]"

Output:

Multiplication of two number : 30

Arguments with default values

```
proc employee_modifier {emp_ID {code 000}} {  
    return [join [list $code $emp_ID] ""]  
}  
puts [employee_modifier 7892]
```

Output : 0007892

Procs with variable arguments

```
set employees [list "raju" "danish" "ratnesh" "ekansh"]

proc email {employee_names} {
    set empID {}
    foreach name $employee_names {
        set ID [join [list $name "@ymail.com"] ""]
        lappend empID $ID
    }
    return $empID
}

puts [email $employees]
```

Output:

 raju@ymail.com danish@ymail.com ratnesh@ymail.com ekansh@ymail.com 

Example : Automatic email address generator using proc.

Input:

set name [list moied samin sharif safa mamun shayok]

```
proc e_id {em_name} {
    set post_fix "@ulkasemi.com"
    set i 250120
    foreach in $em_name {
        set email [join [list $in$i$post_fix]]
        puts $email
        set i [expr $i + 1]
    }
}
```

puts [e_id \$name]

Output:

moied250120@ulkasemi.com
samin250121@ulkasemi.com
sharif250122@ulkasemi.com
safa250123@ulkasemi.com
mamun250124@ulkasemi.com
shayok250125@ulkasemi.com

Nested Lists in TCL